

Guía de Estudio — Examen Final Transversal (EFT)

ITY1101 Gestión de Datos para IA · Entrega/defensa: 13-07-2026 · 40% del ramo

Complementa la [Guía de la Ev3](#) (modelo, métricas, seguridad, demo). Esta guía cubre **lo nuevo del EFT**: infraestructura, monitoreo, gobernanza, despliegue y PMBOK.

1. Cómo se gana la nota

Componente	Peso	Dónde está
Encargo (informe)	15% grupal	docs/Informe_EFT.pdf (12 págs) ✓
Presentación + demo	15% grupal	Deck 15 láminas + demo en vivo ✓
Preguntas individuales	70%	Esta guía + la de Ev3

Indicadores que más pesan en preguntas: **#7 procesos DataOps (20%) · #8 métricas (20%) · #9 seguridad/ley (20%) · #10 implementar mejoras (10%)**. Los tres primeros ya los dominan (guía Ev3); el #10 y los temas EFT están abajo.

2. Requerimiento de infraestructura (slide 12 · informe §5)

Discurso de 30 segundos: *"Dimensionamos la infraestructura con mediciones reales, no supuestos: el sistema completo usa ~200 MB de RAM y 10% de CPU para 7.043 registros, así que en producción pedimos 1–2 vCPU y 2–4 GB por servicio —margen de 10×—. Elegimos nube porque el costo inicial es cero, el cómputo es elástico (entrenar toma segundos, no justifica hardware dedicado) y los servicios gestionados incluyen backups y TLS. Ya operamos 100% en nube, la decisión está validada."*

Preguntas probables:

- **¿Por qué nube y no on-premise?** → Costo inicial cero, elasticidad, servicios gestionados. On-premise solo se justificaría por una política estricta de soberanía de datos; la Ley 21.719 no lo exige si se cumplen sus garantías.
- **¿Cómo escala si los datos se multiplican por 100?** → El predictor es *stateless* (el modelo vive en la BD): se replica horizontalmente tras un balanceador. El pipeline procesa por lotes: se particiona la carga. Para millones de registros, sumar un orquestador (Airflow) manteniendo la misma lógica por etapas.
- **¿Dónde está la redundancia?** → El único punto único de fallo real es la BD → réplica de lectura + backups diarios verificados. Los servicios tienen health checks + reinicio automático; ≥ 2 réplicas del predictor dan 99,9% de disponibilidad.
- **¿Qué dependencia es crítica fijar y por qué?** → [scikit-learn 1.9](#): el modelo serializado exige la **misma versión** al deserializar. (Lo vivimos: un numpy sin fijar nos tumbó el dashboard con un segfault — por eso

todo va pinneado.)

- **¿Por qué misma región para servicios y BD?** → Nuestro benchmark mostró que la **red domina** (~219× más lenta que local): colocalizar reduce el cuello de botella real.

3. Estrategia de monitoreo (slide 13 · informe §6)

Discurso de 30 segundos: "Monitoreamos en tres planos. Infraestructura: ¿está viva la solución? — *health checks* con reinicio automático hoy, Prometheus+Grafana con alertas propuesto. Datos: ¿el pipeline procesa bien? — *carga_logs* audita cada corrida hoy, alerta si los rechazados superan el 5% propuesto. Y modelo: ¿sigue prediciendo bien? — *métricas versionadas con el artefacto* hoy, monitoreo de deriva y recall mensual propuesto. El plano del modelo es el más importante: un modelo degradado falla en silencio."

Preguntas probables:

- **¿Qué es la deriva (drift) y cómo la detectas?** → Los datos de producción dejan de parecerse a los de entrenamiento (ej.: cambia el mix de contratos tras una promoción). Se detecta comparando distribuciones por variable (PSI) y midiendo el recall sobre churn real observado cada mes. Si recall < 70% → re-entrenar.
- **¿Qué alertas configurarías?** → Infra: latencia p95 > 2 s o errores 5xx > 1% → Slack. Datos: rechazados > 5% o duración > 2× histórico. Modelo: PSI alto o recall < umbral → disparar re-entrenamiento.
- **¿Y Jenkins?** → Su rol (CI) lo cubre **GitHub Actions**: pytest corre en cada push. Prometheus/Grafana quedan propuestos porque su valor aparece con operación 24/7.
- **¿Qué monitoreo ya funciona hoy en tu demo?** → `/health` en cada microservicio (Railway lo usa para reiniciar ante fallo), `carga_logs` con leídos/rechazados/duración por corrida, y el dashboard se auto-refresca reflejando el estado real de la BD cada 3 s.

4. Seguridad y gobernanza (slide 9 · informe §7)

Seguridad ya la dominan (guía Ev3). Lo nuevo es **gobernanza** — la diferencia conceptual:

Seguridad = proteger los datos (RLS, TLS, secretos, privilegio mínimo). **Gobernanza** = gestionar los datos como activo: quién accede a qué (políticas por rol), de dónde viene cada dato (linaje raw→clean→validated→BD), qué modelo estaba en producción y cuándo (versionado en `modelo_artefacto`), cómo se ejercen los derechos del titular (ARCO por `customer_id`), y qué riesgos hay y cómo se mitigan (matriz).

Preguntas probables:

- **¿Cómo garantizas trazabilidad/linaje?** → Zonas del pipeline con timestamp por corrida + `carga_logs` + git. Puedo reconstruir el camino de cualquier registro desde el CSV crudo hasta la predicción.
- **¿Cómo respondes a un cliente que pide borrar sus datos (ARCO)?** → Todo está indexado por `customer_id`: se localiza y elimina/rectifica en `clientes` y `predicciones`. La minimización ayuda: no guardamos nombre ni RUT.

- **¿Quién puede ver las predicciones?** → Separación de funciones: el pipeline (dueño) escribe; BI consume con rol de **solo lectura** (`telco_lectura`); `anon` está bloqueado por RLS.
-

5. Estrategia de despliegue organizacional (slide 14 · informe §8)

Discurso de 30 segundos: "Proponemos adopción progresiva en 4 fases para minimizar impacto: piloto con el equipo de retención usando el dashboard contra un grupo control; integración al CRM vía API —la ficha del cliente consulta `/predict/nuevo` y muestra riesgo y factores en vivo—; automatización del scoring semanal alimentando campañas; y operación continua con re-entrenamiento y monitoreo de deriva. Cada fase tiene criterio de avance medible, y el rollback es inmediato porque el modelo queda versionado en la BD."

Preguntas probables:

- **¿Cómo se integra con los sistemas existentes?** → La **API REST es el contrato**: el CRM consume `POST /predict/nuevo` (predicción al vuelo, con factores explicativos para el ejecutivo en llamada); el batch semanal actualiza `predicciones` para los tableros. No se toca el core del CRM.
 - **¿Y la migración de datos?** → Solo cambia la **etapa de ingesta** (de CSV a la fuente transaccional); limpieza, validación y carga quedan igual — esa es la gracia del diseño por etapas.
 - **¿Capacitación?** → Por rol: ejecutivos (leer dashboard + el "porqué" de cada predicción), analistas (interpretar métricas), TI (operación y alertas).
 - **¿Qué pasa si el modelo nuevo es peor?** → *Rollback* inmediato: `modelo_artefacto` conserva versión, métricas y fecha; el predictor recarga el anterior con `/reload`.
 - **¿Beneficio concreto?** → Con recall 79,7%, de cada 100 fugas anticipamos ~80. Retener incluso una fracción paga con holgura la operación (la infra es de bajo costo).
-

6. Metodología PMBOK y plan de gestión (informe §2 — indicadores 1 y 2)

Respuesta modelo: "Usamos PMBOK híbrido: ciclo predictivo para la estructura —alcance y hitos estaban fijos (las evaluaciones)— y desarrollo adaptativo en sprints semanales dentro de cada fase, porque los requisitos evolucionaron con el feedback del docente: tras la Parcial 2 desacoplamos el monolito en microservicios; en la Parcial 3 separamos entrenamiento de inferencia. Ejemplo concreto: el hito 'Parcial 3' fijaba el entregable, pero el cómo se iteró — baseline → balanceado → serving. Aplicamos los 5 grupos de procesos: inicio (caso de negocio del churn), planificación (EDT por fases + matriz de riesgos), ejecución (sprints + commits descriptivos), monitoreo y control (CI con pytest + revisión del docente por parcial), cierre (este EFT). GitHub fue la herramienta de gestión — commits documentan sprints, releases los hitos—; en una organización escalaríamos a Jira."

Riesgos gestionados (si piden ejemplos): suspensión del free tier (mitigación: verificación pre-demo), sobreajuste (holdout estratificado), fuga de credenciales (.env + escaneo del historial).

7. Indicador 10: "¿Cómo implementarías las mejoras?" (10%)

Respuesta con **plan claro y realista** (no solo enumerar):

1. **Ajuste del umbral de decisión** (1 día): barrido del umbral sobre el holdout optimizando F1 o costo de negocio; sube precisión sin reentrenar. *Primera porque es la de mayor impacto/costo.*
2. **Validación cruzada k-fold** (1 día): `cross_val_score` (k=5) en el modo `--eval`; da intervalos de confianza de las métricas con dataset pequeño.
3. **Regularizar el Random Forest** (2 días): búsqueda de hiperparámetros (`max_depth` , `min_samples_leaf`) con CV; cierra la brecha train/test.
4. **Actualizar dependencias con CVEs** (1 día): python-dotenv inmediato; FastAPI/starlette coordinado con re-test de la API.
5. **Monitoreo de deriva** (1 semana): job mensual que calcula PSI y recall contra churn observado; alerta y re-entrena.

Nota honesta que suma puntos: varias mejoras propuestas en parciales **ya las implementamos**: microservicios train/predict separados, rol de solo lectura, y predicción de clientes nunca vistos con factores explicativos (`/predict/nuevo`).

8. Guión sugerido de presentación (15 min, 2 personas, deck de 12 láminas)

Min	Láminas	Quién	Contenido (indicador que cubre)
0–2	1–2	A	Portada · pipeline Fase 1 + DataOps vs tradicional (#4, #7)
2–4	3–4	A	Reto de negocio (#4) · comparación de modelos (#8) — decir verbal: partición 70/30 estratificada y <code>class_weight='balanced'</code>
4–6	5–6	B	Métricas e interpretación (#8) · hallazgo overfitting (#6)
6–7	7	B	Seguridad y gobernanza: ley + técnicas (#5, #9)
7–10	8	A+B	DEMO (instrucción B.c): vaciar → entrenar → predecir → dashboard → cliente nuevo (84% vs 1,4%)
10–13	9–11	B	Infraestructura · monitoreo · despliegue (criterios D del EFT) — decir verbal en infra: el benchmark ~219× justifica "misma región"
13–15	12	A	Limitaciones, mejoras y cierre (#6, #10)

Recortado a 12 láminas: se quitaron Análisis de datos, Diseño del modelo y Rendimiento en la nube (no exigidas explícitamente). Su contenido se dice **verbalmente** donde se indica arriba — sigue siendo materia de preguntas.

Checklist pre-defensa: servicios despiertos (`/health` de los 3 + dashboard), deck en Vercel carga, `checklist_defensa.md` de la Ev3 sigue vigente.